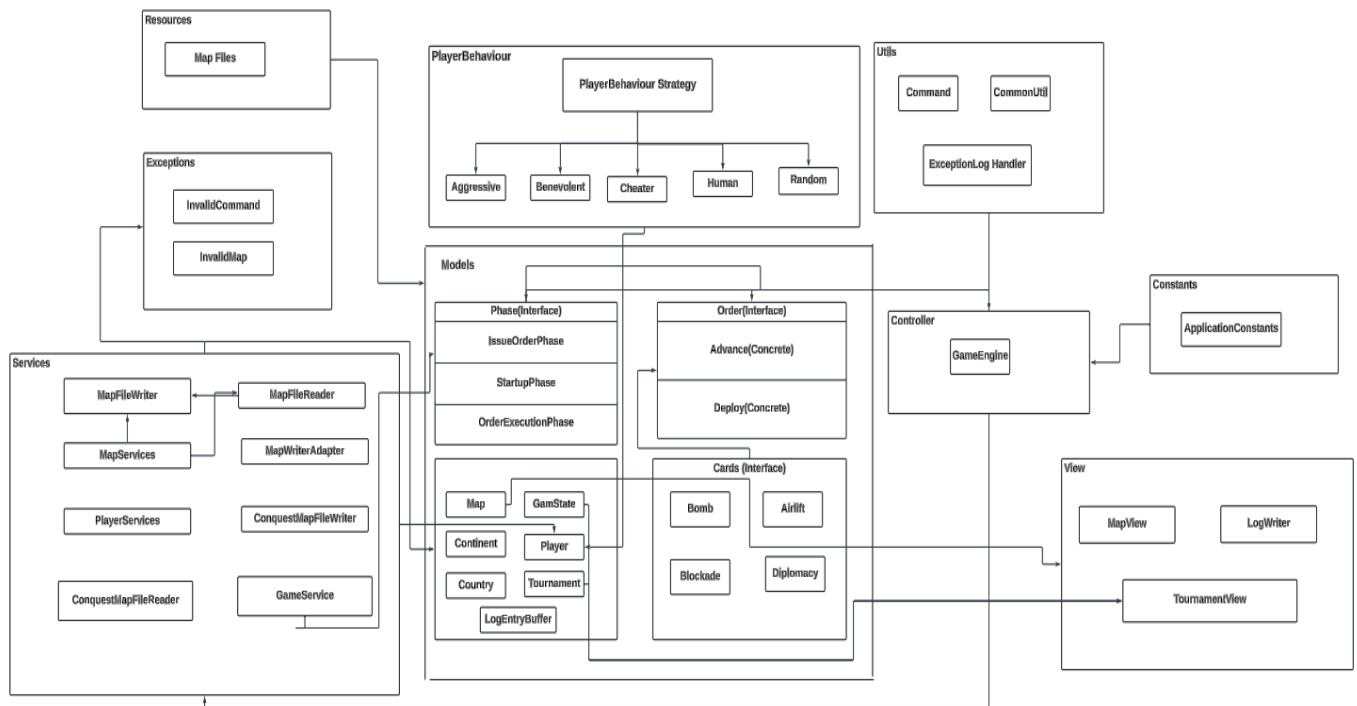


SOEN 6441 Advanced Programming Languages Winter (2024)

Team 09 - Build # 3 Architecture Diagram



In the provided architectural design, the following design patterns are incorporated:

1. State Pattern:

- Phase: Within the Models package, the 'Phase' class hierarchy implements the State pattern. It encapsulates the behavior of different phases of the game, such as IssueOrderPhase, StartupPhase, and OrderExecutionPhase.

2. Command Pattern:

- Order: In the Models package, the 'Order' class represents commands issued by players. It encapsulates actions to be executed, such as advancing armies or deploying armies, thus implementing the Command pattern.

3. Observer Pattern:

- LogEntryBuffer: This class within the Models package acts as the observer, recording game logs. It observes various stages and actions in the game, reflecting the Observer pattern implementation.

4. Adapter Pattern:

- MapWriterAdapter and MapReaderAdapter: These classes in the Services package act as adapters to write and read map files respectively. They encapsulate the logic to interface with different map file formats, such as Conquest maps, reflecting the Adapter pattern.

5. Strategy Pattern:

- PlayerBehaviorStrategy: This class hierarchy within the Models package represents different player behaviors, such as AggressivePlayer, BenevolentPlayer, etc. Each strategy encapsulates a distinct behavior, demonstrating the Strategy pattern implementation.

The Packages used in this complete architecture are explained below:

i) Models Package:

- Player: This class encapsulates data and behavior related to individual players in the game, managing their assets and actions.
- Order: Responsible for processing and executing orders issued by players, including movement of armies between countries.
- GameState: Manages the current state of the game, including players, their orders, and pending actions yet to be executed.
- Map: Handles the structure of the game map, ensuring connectivity between countries and continents, and validating the overall map configuration.

ii) View Package:

- MapView: Provides a visual representation of the game map, allowing players to see the current state of territories and borders.
- LogWriter: Manages the recording of game logs, storing information about various stages and actions taken during gameplay.
- TournamentView: Displays the progress and outcomes of tournament gameplay, providing an overview of matches and results.

iii) Controller Package:

- GameEngine: Acts as the central hub for processing player commands and controlling the flow of the game, orchestrating interactions between different components.

iv) Services Package:

- MapServices: Handles operations related to the game map, such as loading, editing, and saving map configurations.
- PlayerServices: Provides functionalities for managing players, including adding or removing them, assigning territories, and distributing armies.
- GameServices: Offers services related to the overall game, such as saving and loading game states for persistence across sessions.

v) Utils Package:

- Command: Parses and formats commands issued by players, ensuring they are correctly interpreted by the system.
- CommonUtil: Provides utilities for handling input files and performing common tasks within the application.
- ExceptionLogHandler: Manages exceptions related to logging, ensuring that errors are properly captured and handled.

vi) Constants Package:

- ApplicationConstants: Contains static members used throughout the application, providing a centralized location for common values and configurations.

vii) Exceptions Package:

- InvalidCommand: Handles situations where players input commands that are not recognized or permissible within the game context.
- InvalidMap: Deals with errors arising from attempts to create or load invalid map configurations, providing informative feedback to the user.

viii) Resources:

- Map Files: Stores and manages map files used by the game, allowing players to create, edit, and load custom map configurations.